

Student Management System

Database Schema Documentation

Total Tables	12
Database Type	Relational (SQL)
Core Entities	Students, Teachers, Courses, Departments, Classes
Relationship Tables	Enrollments, Attendance, Exams, Results, Fees, Parents, Timetable

Table of Contents

1. Overview & Entity Summary
2. Students
3. Teachers
4. Departments
5. Courses
6. Classes
7. Enrollments
8. Attendance
9. Exams
10. Results
11. Fees
12. Parents
13. Timetable
14. Relationship Diagram (Text Form)
15. Table Creation Order

1. Overview & Entity Summary

This document describes the complete relational schema for the Student Management System (SMS). The database consists of 12 tables split into two categories: **Core Entity Tables** (Students, Teachers, Departments, Courses, Classes) that hold master records, and **Relationship / Transactional Tables** (Enrollments, Attendance, Exams, Results, Fees, Parents, Timetable) that connect core entities and record activity over time.

#	Table Name	Purpose	Type
1	Students	Stores student personal & admission details	Core
2	Teachers	Stores teacher / faculty details	Core
3	Departments	Academic departments (e.g. Computer Applications)	Core
4	Courses	Subjects / courses offered under a department	Core
5	Classes	Class or section groupings (e.g. BCA 1st Year)	Core
6	Enrollments	Links students to courses & classes	Relationship
7	Attendance	Daily attendance records per student per class	Relationship
8	Exams	Exam schedule per course	Relationship
9	Results	Student marks & grades per exam	Relationship
10	Fees	Fee payment tracking per student	Relationship
11	Parents	Parent / guardian details linked to students	Relationship
12	Timetable	Weekly class schedule (day/time/teacher/course)	Relationship

2. Students

Stores personal, contact, and admission information for every student enrolled in the institution. This is a core master table referenced by nearly all relationship tables.

Column Structure

Column	Type	Key	Description
student_id	INT	PK	Unique identifier, auto-increment
first_name	VARCHAR(50)	-	Student's first name
last_name	VARCHAR(50)	-	Student's last name
gender	VARCHAR(10)	-	Male / Female
dob	DATE	-	Date of birth
email	VARCHAR(100)	-	Contact email (should be unique)
phone	VARCHAR(15)	-	Contact phone number
address	VARCHAR(150)	-	Residential address
admission_date	DATE	-	Date the student was admitted

SQL Definition

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  gender VARCHAR(10),  
  dob DATE,  
  email VARCHAR(100),  
  phone VARCHAR(15),  
  address VARCHAR(150),  
  admission_date DATE  
);
```

Foreign Key Notes: None (this is the top-level parent table for the Student side of the schema).

3. Teachers

Stores personal and professional details of teaching staff / faculty, including subject specialization.

Column Structure

Column	Type	Key	Description
teacher_id	INT	PK	Unique identifier, auto-increment
first_name	VARCHAR(50)	-	Teacher's first name
last_name	VARCHAR(50)	-	Teacher's last name
email	VARCHAR(100)	-	Contact email
phone	VARCHAR(15)	-	Contact phone number
subject_specialization	VARCHAR(100)	-	Main subject/area of expertise
hire_date	DATE	-	Date joined the institution

SQL Definition

```
CREATE TABLE Teachers (  
  teacher_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(100),  
  phone VARCHAR(15),  
  subject_specialization VARCHAR(100),  
  hire_date DATE  
);
```

Foreign Key Notes: Referenced by Departments (head_teacher_id), Classes (class_teacher_id), and Timetable (teacher_id).

4. Departments

Represents academic departments such as Computer Applications, Business Administration, Library and Information Science, MCA, and Commerce. Each department has a head teacher.

Column Structure

Column	Type	Key	Description
department_id	INT	PK	Unique identifier, auto-increment
department_name	VARCHAR(100)	-	Name of the department
head_teacher_id	INT	FK	References Teachers(teacher_id)

SQL Definition

```
CREATE TABLE Departments (  
  department_id INT PRIMARY KEY AUTO_INCREMENT,  
  department_name VARCHAR(100) NOT NULL,  
  head_teacher_id INT,  
  FOREIGN KEY (head_teacher_id) REFERENCES Teachers(teacher_id)  
);
```

Foreign Key Notes: head_teacher_id -> Teachers.teacher_id. Teachers table must exist before Departments.

5. Courses

Lists all courses/subjects offered, each belonging to exactly one department (e.g. Programming Fundamentals under Computer Applications).

Column Structure

Column	Type	Key	Description
course_id	INT	PK	Unique identifier, auto-increment
course_name	VARCHAR(100)	-	Name of the course
course_code	VARCHAR(20)	-	Short unique course code (e.g. BCA101)
credits	INT	-	Credit hours for the course
department_id	INT	FK	References Departments(department_id)

SQL Definition

```
CREATE TABLE Courses (  
  course_id INT PRIMARY KEY AUTO_INCREMENT,  
  course_name VARCHAR(100) NOT NULL,  
  course_code VARCHAR(20),  
  credits INT,  
  department_id INT,  
  FOREIGN KEY (department_id) REFERENCES Departments(department_id)  
);
```

Foreign Key Notes: department_id -> Departments.department_id. Departments table must exist before Courses.

6. Classes

Represents class/section groupings such as 'BCA - 1st Year' or 'BBA - Final Year', each assigned a class teacher (incharge).

Column Structure

Column	Type	Key	Description
class_id	INT	PK	Unique identifier, auto-increment
class_name	VARCHAR(50)	-	Name of the class (e.g. BCA - 1st Year)
academic_year	VARCHAR(20)	-	Academic year (e.g. 2024-2025)
class_teacher_id	INT	FK	References Teachers(teacher_id)

SQL Definition

```
CREATE TABLE Classes (  
  class_id INT PRIMARY KEY AUTO_INCREMENT,  
  class_name VARCHAR(50) NOT NULL,  
  academic_year VARCHAR(20),  
  class_teacher_id INT,  
  FOREIGN KEY (class_teacher_id) REFERENCES Teachers(teacher_id)  
);
```

Foreign Key Notes: class_teacher_id -> Teachers.teacher_id. Teachers table must exist before Classes.

7. Enrollments

A many-to-many mapping table connecting students to the courses and classes they are enrolled in. This is the central relationship table of the schema.

Column Structure

Column	Type	Key	Description
enrollment_id	INT	PK	Unique identifier, auto-increment
student_id	INT	FK	References Students(student_id)
course_id	INT	FK	References Courses(course_id)
class_id	INT	FK	References Classes(class_id)
enrollment_date	DATE	-	Date the enrollment was recorded

SQL Definition

```
CREATE TABLE Enrollments (  
  enrollment_id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT,  
  course_id INT,  
  class_id INT,  
  enrollment_date DATE,  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id),  
  FOREIGN KEY (class_id) REFERENCES Classes(class_id)  
);
```

Foreign Key Notes: student_id -> Students.student_id, course_id -> Courses.course_id, class_id -> Classes.class_id.
All three parent tables must exist first.

8. Attendance

Records daily attendance status (Present / Absent / Late) for each student within a class.

Column Structure

Column	Type	Key	Description
attendance_id	INT	PK	Unique identifier, auto-increment
student_id	INT	FK	References Students(student_id)
class_id	INT	FK	References Classes(class_id)
date	DATE	-	Date of the attendance record
status	ENUM	-	'Present', 'Absent', or 'Late'

SQL Definition

```
CREATE TABLE Attendance (  
  attendance_id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT,  
  class_id INT,  
  date DATE NOT NULL,  
  status ENUM('Present', 'Absent', 'Late') NOT NULL,  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (class_id) REFERENCES Classes(class_id)  
);
```

Foreign Key Notes: student_id -> Students.student_id, class_id -> Classes.class_id. Optional: add UNIQUE(student_id, date) to prevent duplicate same-day entries.

9. Exams

Stores exam details for each course, such as exam name, date, and maximum marks.

Column Structure

Column	Type	Key	Description
exam_id	INT	PK	Unique identifier, auto-increment
course_id	INT	FK	References Courses(course_id)
exam_name	VARCHAR(100)	-	Name of the exam (e.g. Mid Term Exam)
exam_date	DATE	-	Date the exam is/was held
total_marks	INT	-	Maximum marks for the exam

SQL Definition

```
CREATE TABLE Exams (  
  exam_id INT PRIMARY KEY AUTO_INCREMENT,  
  course_id INT,  
  exam_name VARCHAR(100),  
  exam_date DATE,  
  total_marks INT,  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

Foreign Key Notes: course_id -> Courses.course_id. Courses table must exist before Exams.

10. Results

Stores the marks obtained and grade earned by each student for a specific exam.

Column Structure

Column	Type	Key	Description
result_id	INT	PK	Unique identifier, auto-increment
student_id	INT	FK	References Students(student_id)
exam_id	INT	FK	References Exams(exam_id)
marks_obtained	INT	-	Marks scored by the student
grade	VARCHAR(5)	-	Grade derived from marks (e.g. A+, B, F)

SQL Definition

```
CREATE TABLE Results (  
  result_id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT,  
  exam_id INT,  
  marks_obtained INT,  
  grade VARCHAR(5),  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)  
);
```

Foreign Key Notes: student_id -> Students.student_id, exam_id -> Exams.exam_id. Both parent tables must exist first.

11. Fees

Tracks fee payments per student, including due date, actual payment date, and payment status.

Column Structure

Column	Type	Key	Description
fee_id	INT	PK	Unique identifier, auto-increment
student_id	INT	FK	References Students(student_id)
amount	DECIMAL(10,2)	-	Fee amount
due_date	DATE	-	Date the payment is due
payment_date	DATE	-	Date actually paid (NULL if unpaid)
status	ENUM	-	'Paid', 'Unpaid', or 'Partial' (default: Unpaid)

SQL Definition

```
CREATE TABLE Fees (  
  fee_id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT,  
  amount DECIMAL(10,2) NOT NULL,  
  due_date DATE,  
  payment_date DATE,  
  status ENUM('Paid', 'Unpaid', 'Partial') DEFAULT 'Unpaid',  
  FOREIGN KEY (student_id) REFERENCES Students(student_id)  
);
```

Foreign Key Notes: student_id -> Students.student_id. Students table must exist before Fees.

12. Parents

Stores parent / guardian contact details linked to a student. Siblings studying at the same institution can share the same parent name, phone, and email across different student_id rows.

Column Structure

Column	Type	Key	Description
parent_id	INT	PK	Unique identifier, auto-increment
student_id	INT	FK	References Students(student_id)
name	VARCHAR(100)	-	Parent/guardian's full name
relation	ENUM	-	'Father', 'Mother', or 'Guardian'
phone	VARCHAR(15)	-	Contact phone number
email	VARCHAR(100)	-	Contact email

SQL Definition

```
CREATE TABLE Parents (  
  parent_id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT,  
  name VARCHAR(100) NOT NULL,  
  relation ENUM('Father', 'Mother', 'Guardian') NOT NULL,  
  phone VARCHAR(15),  
  email VARCHAR(100),  
  FOREIGN KEY (student_id) REFERENCES Students(student_id)  
);
```

Foreign Key Notes: student_id -> Students.student_id. One-to-many: a student can have multiple parent/guardian rows.

13. Timetable

Defines the weekly class schedule: which course is taught by which teacher, in which class, on which day and time slot.

Column Structure

Column	Type	Key	Description
timetable_id	INT	PK	Unique identifier, auto-increment
class_id	INT	FK	References Classes(class_id)
course_id	INT	FK	References Courses(course_id)
teacher_id	INT	FK	References Teachers(teacher_id)
day_of_week	ENUM	-	Monday through Saturday
start_time	TIME	-	Period start time
end_time	TIME	-	Period end time

SQL Definition

```
CREATE TABLE Timetable (  
  timetable_id INT PRIMARY KEY AUTO_INCREMENT,  
  class_id INT,  
  course_id INT,  
  teacher_id INT,  
  day_of_week ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday') NOT NULL,  
  start_time TIME NOT NULL,  
  end_time TIME NOT NULL,  
  FOREIGN KEY (class_id) REFERENCES Classes(class_id),  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id),  
  FOREIGN KEY (teacher_id) REFERENCES Teachers(teacher_id)  
);
```

Foreign Key Notes: class_id -> Classes.class_id, course_id -> Courses.course_id, teacher_id -> Teachers.teacher_id. Optional: add UNIQUE(teacher_id, day_of_week, start_time) to prevent scheduling clashes.

15. Table Creation Order

Because of foreign key dependencies, tables must be created in the order below to avoid 'reference does not exist' errors. Parent tables always come before the child tables that reference them.

Step	Table	Depends On
1	Teachers	None
2	Students	None
3	Departments	Teachers
4	Courses	Departments
5	Classes	Teachers
6	Enrollments	Students, Courses, Classes
7	Attendance	Students, Classes
8	Exams	Courses
9	Results	Students, Exams
10	Fees	Students
11	Parents	Students
12	Timetable	Classes, Courses, Teachers

Document generated for the Student Management System project. This schema reflects the 12-table design covering students, teachers, courses, departments, classes, enrollments, attendance, exams, results, fees, parents, and timetable.